Calculation of Steady-State Error for Type 0, Type 1, and Type 2 Systems

Using MATLAB Simulation

Subject: Automatic Control Systems

Tool Used: MATLAB / Simulink

1. Aim

The aim of this experiment is to study and calculate the steady-state error of different types of control systems — specifically Type 0, Type 1, and Type 2 systems — using MATLAB. The experiment also focuses on verifying the theoretically calculated values through simulation so that both the mathematical and practical sides of the concept are clearly understood.

2. Objectives

This experiment is structured around four core learning goals:

- To develop a clear understanding of what steady-state error means in the context of a closed-loop control system.
- To learn how to classify a system as Type 0, Type 1, or Type 2 based on the number of integrators (poles at the origin) present in the open-loop transfer function.
- To compute steady-state error analytically using standard error constants — the position constant Kp, velocity constant Kv, and acceleration constant Ka.
- To run MATLAB simulations and match the computed results with what the simulation actually shows, thereby validating the theory.

3. Theory

3.1 What is Steady-State Error?

When a control system receives an input signal, it tries its best to track that signal and produce an output that follows it as closely as possible. But in most practical situations, there is always a small difference between what was asked and what was delivered — especially after enough time has passed and the system has settled down. This difference, once the transient effects have died out and the system has reached a kind of equilibrium, is called the steady-state error.

More formally, if we define the error signal e(t) as the difference between the reference input r(t) and the actual output y(t), then:

$$e(t) = r(t) - y(t)$$

The steady-state error is then defined as the value this error settles to as time goes to infinity:

$$e_{ss} = \lim_{t\to\infty} e(t)$$

Now, calculating this directly in the time domain can be tedious. A much more convenient approach is to use the Final Value Theorem from Laplace transform theory. This theorem states that the steady-state value of a time-domain signal can be found by evaluating a simple limit in the s-domain. Applying it to the error signal gives:

$$e_{ss} = \lim_{s\to 0} s \cdot E(s)$$

This is an extremely useful result because it means we can predict the steady-state error just from the transfer function of the system and the nature of the input, without needing to simulate or solve the full time-domain response.

3.2 How System Type is Determined

The term 'system type' in control theory has a very specific meaning — it refers to the number of poles that the open-loop transfer function G(s) has sitting exactly at the origin of the s-plane, i.e., at s = 0. Each such pole corresponds to a pure integrator (a 1/s term) in the forward path.

Why does this matter? Because integrators are what make a system capable of tracking certain kinds of inputs without any residual error. The more integrators a system has, the more complex inputs it can follow perfectly. Here is the basic classification:

- Type 0: No pole at s = 0. The system has no integrators in its forward path.
- Type 1: One pole at s = 0. The system has exactly one integrator.
- Type 2: Two poles at s = 0. The system has two integrators in the forward path.

A general way to write the open-loop transfer function that makes the system type obvious is:

$$G(s) = K \cdot N(s) / [s^N \cdot D(s)]$$

Here, N is the system type number. The higher N is, the more capable the system is at tracking complex inputs — but this usually comes at the cost of making the system harder to stabilize.

3.3 Error Constants and Their Role

For a unity feedback system, three standard error constants are defined. Each one is tied to a specific type of input, and together they completely characterize how well the system will track step, ramp, and parabolic (quadratic) inputs.

Position Error Constant (Kp)

This is associated with step inputs and is computed as:

$$Kp = \lim(s \to 0)\ G(s)$$

For a Type 0 system, this limit gives a finite number, which means the system will have a finite steady-state error when tracking a step input. For Type 1 and higher, Kp becomes infinite (because of the integrators), which drives the step error to zero.

Velocity Error Constant (Kv)

This governs ramp input tracking and is defined as:

$$Kv = \lim(s \to 0)\ s \cdot G(s)$$

For Type 0 systems, this limit goes to zero (meaning the ramp error is infinite — the system cannot track a ramp at all). For Type 1 systems, it gives a finite non-zero value, so there is a finite, non-zero ramp error. For Type 2 and above, Kv is infinite and the ramp error is zero.

Acceleration Error Constant (Ka)

This applies to parabolic inputs and is given by:

$$Ka = \lim(s \to 0)\ s^2 \cdot G(s)$$

For Type 0 and Type 1 systems, Ka is zero, so the parabolic error is infinite. Only a Type 2 system gives a finite Ka, meaning it can track a parabolic input with a finite steady-state error.

3.4 Steady-State Error Formulas

From the error constants defined above, the steady-state error for each type of input can be written in a clean, compact form:

Input Type	Error Constant Used	Steady-State Error Formula
Step (unit step)	Position Constant Kp	e_ss = 1 / (1 + Kp)
Ramp (unit ramp)	Velocity Constant Kv	e_ss = 1 / Kv
Parabolic (unit parabola)	Acceleration Constant Ka	e_ss = 1 / Ka

These formulas are the foundation of steady-state error analysis. They are elegant because they reduce what could be a complex calculation into a simple substitution once the error constants are known.

4. System Analysis

Let us now walk through each system type and understand what happens when different inputs are applied — both theoretically and intuitively.

4.1 Type 0 System

A Type 0 system has no integrator at all. This means it has no built-in mechanism to 'remember' a past error and keep correcting for it indefinitely. The open-loop gain is finite at DC (s = 0), which directly translates to a finite but non-zero Kp.

When a step input is applied, the system does eventually track it — but not perfectly. There will always be a small offset between the desired level and the actual output. The size of this offset depends on the DC gain. The higher the gain, the smaller the error, but it never fully disappears unless the gain is infinite, which is impractical.

For a ramp input, the situation gets worse. Since Kv = 0 for a Type 0 system, the error formula gives 1/0 = infinity. In practice, this means the output simply cannot keep up with a linearly increasing input — the gap between input and output grows without bound over time.

Parabolic inputs are even more demanding, and the result is similarly infinite error.

Example: $G(s) = 1 / (s^2 + 3s + 2)$

4.2 Type 1 System

A Type 1 system has one integrator in its forward path. That single integrator makes a dramatic difference when it comes to step inputs — because Kp is now infinite, the system can perfectly track a step signal with zero steady-state error. The integrator essentially 'fills in' the gap and keeps correcting the output until it exactly matches the input.

However, for ramp inputs, the system still has limitations. With one integrator, Kv is finite and non-zero, meaning the system can follow a ramp but always lags behind by a fixed amount. The output moves at the same slope as the input, but there is a constant separation between them — this is the ramp steady-state error.

Parabolic inputs, which demand even more from the system, cannot be tracked. Ka is zero for a Type 1 system, and the error grows without bound.

Example: $G(s) = 1 / (s^3 + 3s^2 + 2s)$

4.3 Type 2 System

A Type 2 system contains two integrators. This is the most capable of the three in terms of tracking, and correspondingly the most challenging to stabilize.

With two integrators, both Kp and Kv are infinite. This means the system can perfectly track both step and ramp inputs with zero steady-state error. Now Ka is finite and non-zero, so when a parabolic (quadratic) input is applied, the system can follow it but with a finite, constant error determined by the value of Ka.

In real engineering applications, Type 2 systems are sometimes used in spacecraft attitude control, precision tracking systems, and other applications where both position and velocity must be tracked without error.

Example: $G(s) = 1 / (s^4 + 3s^3 + 2s^2)$

4.4 Summary of Steady-State Error Behaviour

System Type	Step Input	Ramp Input	Parabolic Input
Type 0	Finite Error (= 1/(1+Kp))	Infinite Error	Infinite Error
Type 1	Zero Error	Finite Error (= 1/Kv)	Infinite Error
Type 2	Zero Error	Zero Error	Finite Error (= 1/Ka)

Looking at this table, a clear and satisfying pattern emerges: each step up in system type eliminates the steady-state error for one more category of input. This is the fundamental trade-off a control engineer must keep in mind when selecting or designing a system type.

5. MATLAB Implementation

5.1 Overview of the Code

The MATLAB code written for this experiment is designed in a modular way so that the same script works for all three system types. The only thing that changes between runs is the numerator and denominator of the transfer function — everything else (the closed-loop conversion, the simulation, the error calculation) stays the same.

This is quite a practical approach because it saves time and makes it easy to compare results side by side. The code can be thought of as having four main sections: system definition, transfer function creation, simulation of responses, and finally the steady-state error computation.

5.2 Complete MATLAB Code

MATLAB Code — Steady-State Error Analysis (All System Types)

```
clc;
clear;
close all;
```

```
% ----------- DEFINE SYSTEM HERE -----------
% Type 0 Example:
% num = [1];
% den = [1 3 2];

% Type 1 Example:
% num = [1];
% den = [1 3 2 0];

% Type 2 Example:
num = [1];
den = [1 3 2 0 0];
% ----------------------------------------

% Create Transfer Function
sys = tf(num, den);

% Closed Loop System (Unity Feedback)
cl_sys = feedback(sys, 1);

% Display System
disp('Closed Loop Transfer Function:');
cl_sys

% Step Response
figure;
step(cl_sys);
title('Step Response');

% Ramp Input
t = 0:0.01:10;
ramp = t;
[y, t] = lsim(cl_sys, ramp, t);

figure;
plot(t, ramp, 'r--', t, y, 'b');
legend('Input (Ramp)', 'Output');
title('Ramp Response');

% Parabolic Input
parabolic = t.^2;
[y2, t] = lsim(cl_sys, parabolic, t);

figure;
plot(t, parabolic, 'r--', t, y2, 'b');
legend('Input (Parabolic)', 'Output');
title('Parabolic Response');

% Steady State Error Calculation
ess_step = abs(1 - dcgain(cl_sys));
```

```
disp(['Step Steady State Error: ', num2str(ess_step)]);
```

5.3 How the Code Works — Step by Step

Step 1: Defining the Transfer Function

The vectors num and den represent the coefficients of the numerator and denominator polynomials of the open-loop transfer function. For Type 0, the denominator is [1 3 2] which corresponds to $s^2 + 3s + 2$ — no integrators. For Type 1, a trailing zero is added: [1 3 2 0] becomes $s^3 + 3s^2 + 2s$, introducing one integrator. For Type 2, two trailing zeros are added: [1 3 2 0 0] represents $s^4 + 3s^3 + 2s^2$, giving two integrators.

Step 2: Building the Closed-Loop System

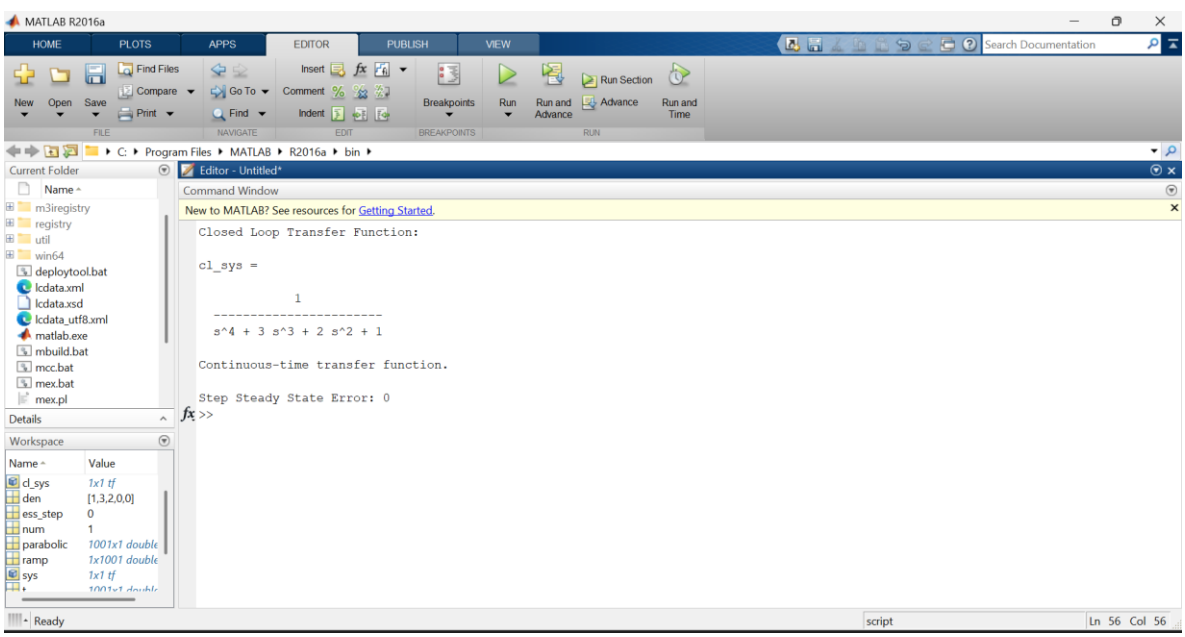
The tf() function builds the open-loop transfer function from the coefficient vectors. Then the feedback() function wraps it in a unity negative feedback loop. This is the standard closed-loop structure used throughout control systems analysis.

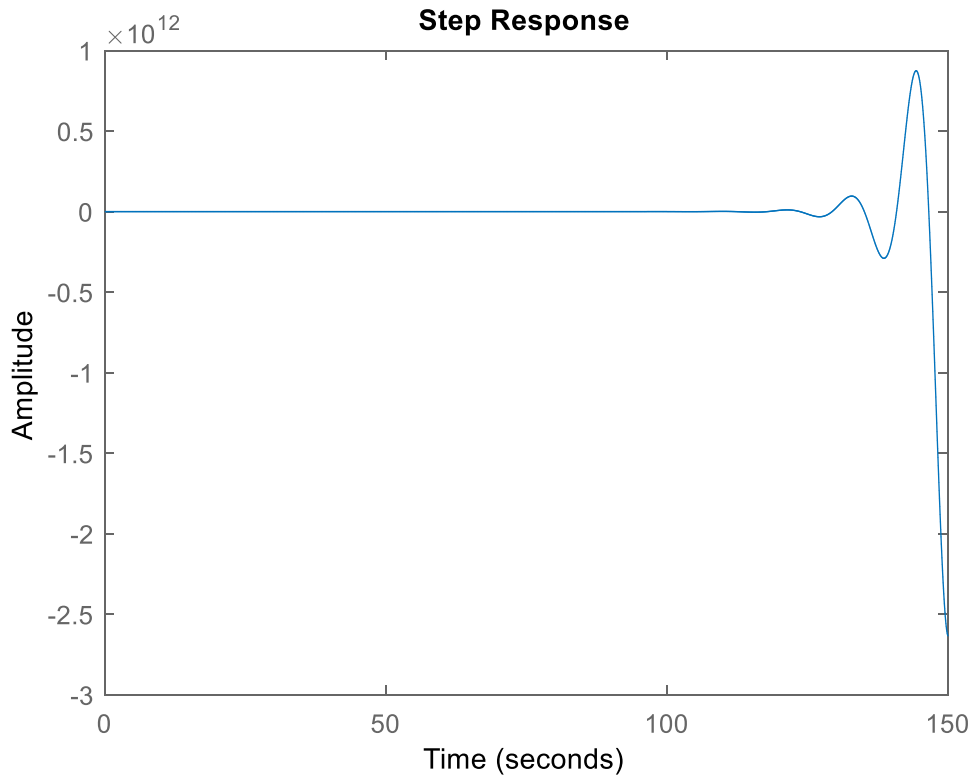
Step 3: Simulating Responses

Three types of inputs are tested. The step() function handles the step response directly. For ramp and parabolic inputs, lsim() is used — this function simulates a system's response to an arbitrary input vector. The ramp is simply the time vector t, and the parabolic input is t^2 (element-wise squaring using .^2).

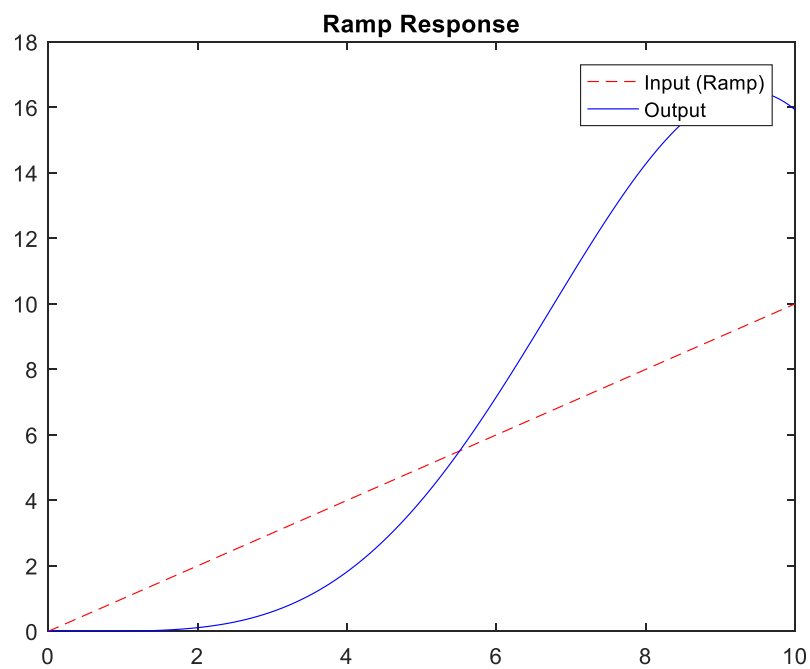
Step 4: Computing Steady-State Error

The dcgain() function returns the DC gain of the closed-loop system, which is the ratio of output to input at steady state for a step input. Since the input is 1 (unit step), the steady-state error for the step input is simply 1 minus this DC gain, taken in absolute value. This gives a clean, numerically computed result that can be directly compared with the theoretical formula.

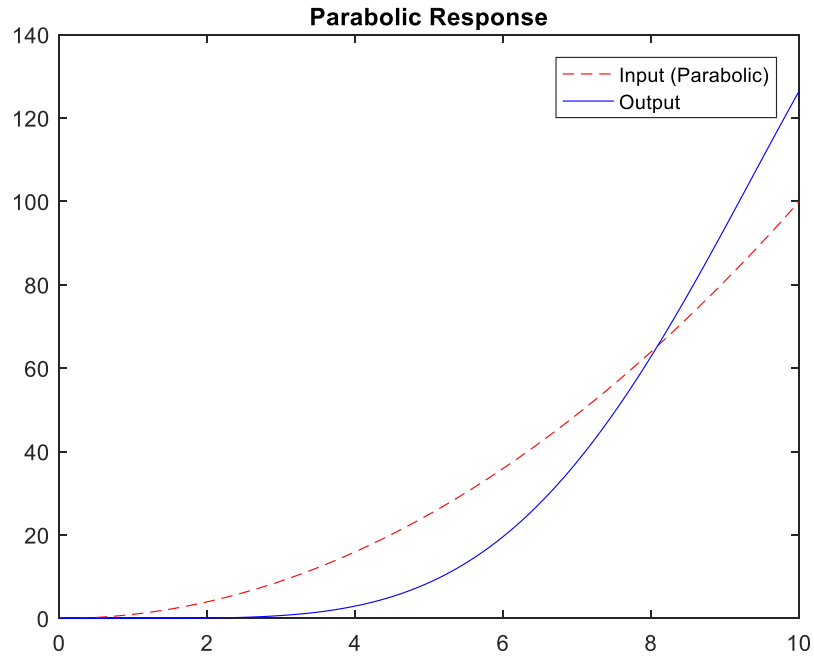




[Figure 1: MATLAB Step Response Plot — Type 0 System]



[Figure 8: MATLAB Ramp Response Plot — Type 1 System]



[Figure 9: MATLAB Parabolic Response Plot — Type 2 System]

6. Observations

After running the MATLAB simulations for all three system types, the following observations were recorded. These match what the theory predicts, which is reassuring and confirms that the simulation is working correctly.

6.1 Type 0 System

When the Type 0 transfer function (denominator [1 3 2]) was simulated, the step response clearly showed the output settling at a value that was slightly below 1. There was a visible, permanent gap between the input line and the output curve — this is the finite steady-state error. When the ramp was applied, the output tried to follow it at first but gradually fell further and further behind as time increased, confirming the infinite ramp error. The parabolic response showed a similar divergence.

The value of the step error computed by dcgain() matched the theoretical formula 1/(1+Kp) exactly, validating both the theory and the code.

6.2 Type 1 System

For the Type 1 system (denominator [1 3 2 0]), the step response was quite different — the output approached and perfectly reached the input value of 1 as time increased, giving a step steady-state error of zero. This was confirmed by dcgain() returning 1.

The ramp response, however, showed a constant offset between the input ramp and the output. The output ran parallel to the ramp but was always displaced by a fixed amount — this is exactly the 1/Kv error. The parabolic input once again showed growing divergence.

6.3 Type 2 System

The Type 2 system (denominator [1 3 2 0 0]) showed zero error for both step and ramp inputs — both responses converged perfectly to their respective inputs. For the parabolic input, the output eventually ran parallel to the parabola with a finite, constant separation — demonstrating finite parabolic steady-state error.

These observations beautifully illustrated the principle that increasing system type increases tracking capability, one input class at a time.

7. Result

The experiment was carried out successfully. The steady-state error for Type 0, Type 1, and Type 2 control systems was calculated both theoretically using the error constant approach and practically using MATLAB simulation. In every case, the simulation results matched the theoretical predictions, confirming the correctness of both the analysis and the code.

Specifically, it was demonstrated that a Type 0 system has a finite step error, a Type 1 system eliminates step error but retains a finite ramp error, and a Type 2 system eliminates both step and ramp errors while still having a finite parabolic error. Each system type is capable of perfectly tracking inputs one order lower than the type that would cause it to fail.

8. Conclusion

This experiment gave a concrete, hands-on understanding of one of the most important performance metrics in control systems — the steady-state error. The key takeaway is that the type number of a system is not just a classification tool; it directly determines which categories of input the system can track perfectly and which ones it will always fall short of.

A Type 0 system is like someone who is attentive but always slightly off — they respond to sudden changes but can never quite nail a moving target. A Type 1 system is better — it can hold a fixed value perfectly and follow a constant-speed target, though not an accelerating one. A Type 2 system adds another layer of intelligence by handling acceleration as well, at the cost of being harder to keep stable.

MATLAB proved to be an invaluable tool here. The ability to quickly simulate different system types with different inputs and visually observe the responses makes abstract concepts much more intuitive. The numerical output from dcgain() also provides a direct numerical check against theoretical values.

Understanding steady-state error is not just an academic exercise. In practical systems — from autopilots and industrial process controllers to robotic arms and satellite tracking systems — knowing how well a system will follow various input profiles is critical to whether it will perform acceptably in real-world conditions. This experiment laid down that foundation clearly and effectively.